

CMSC216: Bonus Review 1B

Chris Kauffman

*Last Updated:
Mon Sep 21 04:05:10 PM EDT 2026*

Bonus Review Rules

- ▶ 2 Questions will be shown with about 5min per question, 10min total, time limit enforced on Gradescope Quiz
- ▶ Each correct answer worth 1 Engagement Point
- ▶ Cooperation is allowed and encouraged within your discussion section - *talk to your neighbors, form a class consensus!*
- ▶ Scores posted after all sections have completed bonus review, likely the following day
- ▶ Review is Open Resource just like the exam:
<https://www.cs.umd.edu/~profk/216/exam-rules.pdf>

Staging

- ▶ Open up the Gradescope Review Quiz for the day
- ▶ Once started, the quiz closes after 10min

Get everybody and their stuff together

Okay...



Problem 1

Which of the following code blocks will correctly read in the count and candidate names of a Tally from a vote data file in Project 1? No error checking is required. Mark ALL codes that are correct.

```
//// A ////
tally_t *tally_from_file_A(char *fname){
    FILE *fin = fopen(fname,"r");
    tally_t *tally = malloc(sizeof(tally_t));
    fscanf(fin, "%d", &tally->candidate_count);
    for(int i=0; i<tally->candidate_count; i++){
        fscanf(fin, "%s", tally->candidate_names[i]);
    }
    ...;
}
```

```
//// C ////
tally_t *tally_from_file_C(char *fname){
    FILE *fin = fopen(fname,"r");
    tally_t *tally = malloc(sizeof(tally_t));
    int count;
    fscanf(fin, "%d", &count);
    for(int i=0; i<count; i++){
        fscanf(fin, "%c", &tally->candidate_names[i]);
    }
    tally->candidate_count = count;
    ...;
}
```

```
//// B ////
tally_t *tally_from_file_B(char *fname){
    FILE *fin = fopen(fname,"r");
    tally_t *tally = malloc(sizeof(tally_t));
    fscanf(fin, "%d", &tally->candidate_count);
    for(int i=0; i<tally->candidate_count; i++){
        fscanf(fin, "%s", &(tally->candidate_names+i));
    }
    ...;
}
```

```
//// D ////
tally_t *tally_from_file_D(char *fname){
    tally_t *tally = malloc(sizeof(tally_t));
    fscanf(fin, "%d", &tally->candidate_count);
    for(int i=0; i<tally->candidate_count; i++){
        fscanf(fname, "%s", tally->candidate_names[i]);
    }
    ...;
}
```

Problem 2

Below is a description of the `node_read()` function and its use in a `main()` function. Mark ALL STATEMENTS that are True about the code.

```
// single data node in linked list
typedef struct node_struct {
    char data[128];
    struct node_struct *next;
} node_t;

node_t *node_read();
// Allocates a node struct and reads string
// data typed by the user into the data
// field of the node. Initializes the next
// pointer to be NULL. Returns a pointer to
// the allocated node.

int main(int argc, char *argv[]){
    node_t *node = node_read();
    printf("node data: %s\n", node->data);
    printf("node next: %p\n", node->next);
    ...
}
```

- ▶ (A) `node_read()` must allocate the node in the Stack
- ▶ (B) `node_read()` must allocate the node in the Heap
- ▶ (C) `main()` must use `free(node)` to de-allocate memory for the struct
- ▶ (D) `node_read()` may use `scanf("%c", ...)` to read typed input
- ▶ (E) `node_read()` may use `fscanf(stdin, "%s", ...)` to read typed input
- ▶ (F) `node_read()` may not set the next field as pointers have the default value NULL